

NEXA / ALS Monitor

Program Overview and Operator Guide

Documentation snapshot: 18 September 2026

1. Purpose and Scope

NEXA is a local Python application that brings together grip-force measurement, eye tracking, shoulder-movement measurement, and speech analysis. The repository is named ALS Monitor; the installed command is `nexa`. A shared PyQt5 interface organizes participants, assessment sessions, results, reports, and baselines. Each measurement service can also run independently from the command line.

This is a research measurement prototype, not a diagnostic medical device. Measurements and changes from baseline are descriptive: they do not establish ALS diagnosis, disease severity, prognosis, or treatment recommendations. Operator supervision, consistent protocols, and independent measurement validation remain necessary. No clinical thresholds are established here.

This document describes the implementation in this repository, not planned features. Hardware-dependent behavior has not all been verified end to end. The operator-reported Raspberry Pi setup is a Pi 4 Model B Rev 1.5, aarch64, Debian 13 (Trixie), with a Waveshare OV9281-110 monochrome CSI camera.

Service Map

- Grip Strength: `als_monitor.grip_monitor`; ESP32 and two load cells; left and right peak force in newtons.
- Oculomotor: `als_monitor.eye_tracker`; Picamera2 CSI capture; visually guided prosaccade latency.
- Motor / Movement: `als_monitor.shoulder_monitor`; webcam and MediaPipe; 2D arm-elevation velocity.
- Speech: `als_monitor.speech_analysis`; microphone or existing WAV; timing, acoustic estimates, and quality flags.
- NEXA UI: `nexa_ui/app.py`; participant/session workflow, review, baseline comparison, and HTML reports.

Guide Contents

1. Purpose and scope
2. Architecture and repository
3. Installation and launching
4. Operator workflow and reports
5. Grip measurement and calibration
6. Eye tracking and CSI camera
7. Shoulder movement
8. Speech assessment
9. Storage, privacy, and baselines
10. Troubleshooting, limitations, and maintenance

2. Architecture and Repository

The UI does not execute all sensor processing inside its own event loop. It launches a modality module as a separate QProcess, merges standard output and standard error into a live log, and reads the service's output artifacts. The service remains responsible for its own capture window and assessment.

```
nexa command / desktop shortcut
-> NEXA UI or standalone service
-> participant and session selection (UI)
-> preparation -> child process -> review
-> save artifacts + metrics -> history / report / baseline
```

Source Layout

scripts/	Linux and Windows launch/install scripts
nexa_ui/	
app.py	Application shell and navigation
screens.py	Participants, sessions, history, system
modalities.py	Service registry and metric definitions
runner.py	Prepare / run / review process controller
database.py	Shared SQLite persistence
report.py	Session HTML report generation
grip_calibration.py	Capture, save, and test calibration
config.py, theme.py	Paths, display, runtime and visual settings
src/als_monitor/	
grip_monitor/	Serial capture and force graph
eye_tracker/	Camera, pupil detection, trials, CSV output
shoulder_monitor/	Pose detection and velocity analysis
speech_analysis/	Recording, WAV analysis, CLI and database
passages/	Rainbow Passage and metadata
data/	Models and standalone service data
nexa_data/	Shared UI database and saved assessments

Each service has a `__main__.py`, enabling `python -m als_monitor.<module>`. The launcher changes to the repository root and adds `src` to `PYTHONPATH`. UI imports such as `import theme` refer to local files, not a package to install.

The runner sets `NEXA_OUTPUT_DIR`, `NEXA_SESSION_ID`, and, when available, `NEXA_PARTICIPANT_ID`. Services write into a per-run staging directory. Saving copies the registered artifacts and `run_log.txt` into a session folder and records a run in SQLite. `Retry/discard` removes the staging directory.

The module registry controls commands, output names, displayed metrics, instructions, and estimated duration. Adding an assessment requires keeping those definitions consistent with the service's actual artifact schema.

3. Installation and Launching

This is a Python project; routine operation does not require compiling the application. Native dependencies such as OpenCV and MediaPipe must nevertheless match the machine's architecture, Python version, and CPU capabilities. The commands below describe the repository's intended setup; package availability depends on the operating system and enabled repositories.

Raspberry Pi

From the project directory, install the system dependencies and create an environment that can access the system Picamera2 packages:

```
sudo apt update
sudo apt install python3-picamera2 python3-opencv python3-numpy \
  python3-pyqt5 python3-pygame python3-matplotlib python3-tk \
  python3-venv python3-serial libportaudio2 alsa-utils
cd "$HOME/Documents/ALS Monitor/als-monitor"
python3 -m venv --system-site-packages .venv
.venv/bin/python -m pip install sounddevice
bash scripts/install-launcher.sh
bash scripts/install-desktop-shortcuts.sh
```

Do not recreate or replace an existing working environment unnecessarily. The Linux installer creates /usr/local/bin/nexa; installation persists across terminal sessions. Moving the repository can require reinstalling the launcher.

```
nexa                # Open the UI
nexa eye            # Prosaccade assessment
nexa preview        # Pupil camera preview
nexa grip           # Grip graph
nexa shoulder       # Shoulder assessment
nexa speech         # Interactive speech menu
nexa help
```

Linux normally uses .venv/bin/python, falling back to python3. For shoulder monitoring, the Linux launcher and UI prefer .venv-shoulder when present. The UI otherwise uses its own interpreter for assessments. See Section 7 for the Pi-specific shoulder environment.

Windows

Use a working Python installation with SQLite support. From PowerShell:

```
py -3 -m pip install numpy opencv-python matplotlib pyserial \
  sounddevice PyQt5 pygame
powershell -ExecutionPolicy Bypass -File .\scripts\install-launcher.ps1
powershell -ExecutionPolicy Bypass -File .\scripts\install-desktop-shortcuts.ps1
```

Open a new terminal after installation and run nexa. The Windows installer places a launcher in the user's local Programs directory and updates user PATH. MediaPipe and its pose model are additional requirements for shoulder capture. Avoid mixing multiple OpenCV wheel variants in the same environment.

The PowerShell launcher checks ALS_MONITOR_PYTHON, an activated environment, python, then py -3, accepting an interpreter that imports SQLite and NumPy. Set ALS_MONITOR_PYTHON to an explicit executable if multiple installations cause confusion. The Windows CLI does not implement the Linux shoulder-specific environment selection; the UI does recognize .venv-shoulder/Scripts/python.exe.

Eye capture currently requires Raspberry Pi Picamera2 and is not available on Windows. Grip needs a Windows serial port such as COM3, configured in the service; the calibration dialog has its own port selection. Speech uses a local microphone, and shoulder uses OpenCV camera index 0.

4. Operator Workflow and Reports

1. Open nexa and use Participants to create or select a participant.
2. Start a session, recording the operator and relevant notes.
3. Select Grip Strength, Oculomotor, Motor / Movement, or Speech.
4. Review the operator checklist and participant instruction, then start.
5. Complete the service's own task; monitor the log for errors.
6. Review metrics and output files. Add notes, save, retry, or discard.
7. Use Assessment results to inspect saved runs and manage baselines.
8. Generate a session report or return to History to review earlier sessions.

The runner has preparation, running, and review phases. It supports aborting a child process and applies a configurable 600-second overall timeout. A process failure is different from a completed assessment with questionable measurement quality: always inspect quality flags, valid samples, and the log.

The UI is designed around 1024x600, can be resized, supports 75%-150% zoom, and uses F11 for fullscreen. Service windows have their own controls and focus. Countdowns are estimates of the overall workflow, not synchronized hardware recording clocks. At zero the UI can show *Finishing* . . . while processing continues. Grip and motor preparation allowances do not equal actual recording duration.

History and Analysis

History provides session reports, assessment results, and access to raw folder locations. Assessment results expose individual runs rather than only the latest summary on each session tile. Details show the metrics retained by the registry; the raw summary file can contain additional fields not presented in the UI.

Shoulder review includes Show analysis for the saved velocity PNG. The current history workflow does not provide that same dedicated plot viewer; the PNG is available in the raw run folder.

Session reports are self-contained HTML files opened with the system's default application. They contain participant/session information, run status, metrics, notes, and speech assessment type. They do not generate diagnoses or reference range interpretations. The PDF accompanying this guide is documentation, not an additional built-in assessment-report export feature.

Baseline Comparison

Select a suitable saved completed run as a baseline, then compare a later run for the same participant and modality. Speech baselines are separated by task: connected speech, sustained vowel, and reading. Comparisons display baseline, current, absolute change, and percentage change where defined. A zero baseline does not support a meaningful percentage change.

The operator must ensure matching units, calibration, protocol, and algorithms. The UI does not automatically establish comparability after software or sensor changes. Old raw-count grip results must not be compared with newton results; old contour-based shoulder results must not be compared with pose results.

5. Grip Measurement and Calibration

The ESP32 sends newline-delimited pairs of raw readings in this order:

```
right_counts, left_counts
```

The default port is `/dev/ttyUSB0` at 115200 baud. The Python service filters invalid/nonfinite readings, converts each hand independently, and plots force against time. It retains left and right maxima and displays elapsed session duration. Closing the graph ends the service; UI launches `produce_grip_summary.json` with peaks, duration, force unit, and calibration values. The graph is not a measurement of a physical Y axis: its vertical axis is the calibrated scalar force from each load-cell channel.

Conversion to Newtons

```
zero_counts = mean unloaded reading
delta_counts = mean known-load reading - zero_counts
scale        = known_mass_kg * 9.80665 / delta_counts
force_N      = abs((raw_counts - zero_counts) * scale * polarity)
```

The calibration retains the signed difference, so either increasing or decreasing sensor counts can represent loading. The final absolute value makes recorded force nonnegative. Polarity is retained in the schema but does not alter the magnitude when it is +1 or -1. A 2 kg reference corresponds to 19.6133 N under the calibration's standard-gravity assumption.

Absolute value also turns reverse loading and baseline noise into positive magnitudes; it does not remove drift, preload, or incorrect mechanical leverage. Validate the complete grip assembly with the load applied along its intended measurement direction. Factory/default counts are not a substitute for calibration.

Built-in fallback values are left zero -114000 with delta 63000 and right zero 59500 with delta -39200, each using a 2 kg reference. Saved per-hand calibration overrides these defaults on service startup.

System Tab Calibration

1. Close any grip service using the serial port, then open System > Calibrate grips.
2. Select the port, hand, and known reference mass.
3. Remove load and capture the unloaded reading.
4. Apply the known load steadily and capture the loaded reading.
5. Save the calibrated hand; the other hand need not be recalibrated.
6. Use Test calibration to inspect live newtons against the expected reference force.

Capture averages fresh serial readings (50 samples), includes a connection settling delay, and rejects an insufficient load difference. Retaring invalidates that hand's unfinished calibration. Saving merges complete hands with existing saved data, preserving the other hand. Test mode can use completed unsaved or saved calibration. Settings are stored in `nexa_data/grip_calibration.json`.

The exported grip summary currently contains peaks and total elapsed duration, not a persistent full force-time trace, contraction duration, or validated fatigue/consistency score. Some checklist text still names Arduino Uno even though the configured deployment uses an ESP32.

6. Eye Tracking and CSI Camera

The eye module selects a unique Picamera2 camera whose model contains ov9281. It does not silently fall back to a USB webcam. The Waveshare OV9281-110 is the identified deployment sensor. Defaults request 640x400 YUV420, 120 FPS, 4000 us manual exposure, and gain 4. Requested frame rate is not guaranteed achieved frame rate; inspect recorded sample intervals.

camera.py extracts grayscale frames and maps sensor timestamps into the monotonic timing domain when available, with receive-time fallback. The sampler feeds pupil observations to the task. pupil_tracker.py uses image thresholding, morphology, and geometric/confidence filtering rather than a learned gaze model.

Preview and Assessment

```
rpicam-hello --list-cameras
nexa preview
nexa eye
```

Preview overlays the pupil position and estimated FPS. B toggles the binary mask; Q or Escape exits. The assessment includes camera feedback and a fixation quality check, followed by 10 trials with targets in left/right/up/down directions. Space advances task stages. Keep head position, lighting, and camera framing stable.

The detector estimates saccade onset using a velocity threshold with an adaptive baseline component, minimum duration/sample count, direction/displacement checks, and latency limits. These are configurable research parameters, not clinical cutoffs. Pupil velocity is in pixels/second, not calibrated gaze degrees/second. Summary statistics include valid/invalid trial counts, mean latency, standard deviation, coefficient of variation, and minimum/maximum latency.

UI runs create eye_results.csv, eye_summary.json, and optionally eye_samples.csv. Important current limitation: the UI registry collects only the first two files, so the sample CSV is not preserved by the normal save flow before staging cleanup. Standalone runs save timestamped trial, sample, and summary CSV files under results/.

CSI Configuration and Current Hardware Status

The deployment configuration uses the following in /boot/firmware/config.txt:

```
[all]
camera_auto_detect=0
dtoverlay=ov9281
```

This Waveshare setup uses the plain OV9281 overlay, not an Arducam-specific option. Shut down and unplug the Pi before touching camera connectors.

As of this documentation snapshot, replacing the ribbon enabled sensor detection, but video capture still timed out in both NEXA and an independent 30 FPS rpicam-still --nopreview test. This remains unresolved; it is not documented as working capture. Hardware, connector, and kernel/driver compatibility require isolation using a known-good camera/Pi or a spare OS card.

The SHARPNESS/no sharpen algorithm message is distinct from a frontend timeout. An invalid DISPLAY value prevents preview rendering but does not explain the observed headless capture timeout. An ov9282 driver label does not by itself mean the selected OV9281 camera model is wrong.

7. Shoulder Movement

The shoulder service uses a USB webcam through OpenCV camera index 0 and MediaPipe Pose Landmarker. The model is loaded from `data/models/pose_landmarker_lite.task`, or the `NEXA_POSE_MODEL` override. It detects anatomical shoulders and wrists, filters low-confidence landmarks, and mirrors the displayed image while retaining anatomical side labels.

Select L or R for the evaluated arm. Press Space or click Start to arm a trial; the service waits for the required landmarks before recording. Keep the selected shoulder and wrist visible. Recording lasts five seconds. Q exits.

The measurement is shoulder-to-wrist elevation relative to image vertical in two dimensions. Angular speed uses absolute angle change over elapsed time, with a three-sample moving average for the peak. Missing tracking resets the previous-angle state. At least three valid velocity samples are required for a result. This is not a 3D shoulder joint angle or a full biomechanical model.

Artifacts are `shoulder_summary.json`, `shoulder_velocity.csv`, and `shoulder_analysis.png`. The plot shows raw angular speed, smoothed speed, and peak reference in deg/s. The service also uses `nexa_evaluations.db`; standalone plot/CSV artifacts are placed under `data/shoulder_monitor/`.

Raspberry Pi-Compatible Runtime

The reported Python 3.13 / MediaPipe 1.0.1 installation terminated with a native AES/illegal-instruction failure on this Pi. A separate Python 3.12 environment with MediaPipe 0.10.14 was reported to run the shoulder service successfully. This is a deployment workaround, not a guarantee for all Pi environments.

```
cd "$HOME/Documents/ALS Monitor/als-monitor"
.venv/bin/python -m pip install uv
.venv/bin/uv python install 3.12
.venv/bin/uv venv --python 3.12 .venv-shoulder
.venv/bin/uv pip install --python .venv-shoulder/bin/python \
  "numpy<2" "opencv-contrib-python<4.12" \
  "mediapipe==0.10.14" matplotlib
```

Download the pose model once if absent:

```
mkdir -p data/models
MODEL_BASE=https://storage.googleapis.com/mediapipe-models/pose_landmarker
MODEL_PATH=pose_landmarker_lite/float16/1/pose_landmarker_lite.task
curl -fL "$MODEL_BASE/$MODEL_PATH" \
  -o data/models/pose_landmarker_lite.task
```

Linux nexa shoulder and motor assessments in the UI select `.venv-shoulder` when its executable exists; other services remain on their normal environment. Close and reopen the UI after updating launcher/configuration code. A missing dedicated environment falls back to the default interpreter, potentially reintroducing the incompatible MediaPipe build.

Keep framing and movement plane consistent across sessions. Out-of-plane motion, occlusion, frame timing, and detection noise can change estimates. Three valid samples are a technical minimum, not evidence of adequate trial coverage.

8. Speech Assessment

The speech UI records new assessments or analyzes existing WAVs without microphone capture. Select the correct task; its label is retained in results and reports.

Tasks

- Connected speech: speak naturally about a familiar topic; UI default 30 seconds.
- Sustained vowel: hold an "ah" at comfortable pitch and volume; UI default 5 seconds.
- Reading: read the bundled Rainbow Passage at a normal pace; UI default 60 seconds.

The passage appears during reading, not preparation. All tasks have operator checklists and instructions. Keep microphone position and room noise consistent.

Recording uses sounddevice, mono PCM16 WAV, and a 180-second cap. It prefers 48000 Hz, with fallback to a supported device default; file headers and analysis use the actual rate. MIC_DEVICE_INDEX selects a nondefault microphone.

Analysis Implemented

The WAV reader accepts 16-bit PCM WAV and averages multichannel input to mono. It calculates recording duration, RMS level in dBFS, clipping fraction, estimated speech duration, internal pause duration/count/percentage, mean fundamental frequency (F0), an autocorrelation-based HNR estimate, and voiced-frame count.

Speech timing uses RMS-energy thresholding in 30 ms frames, removes short speech bursts, and counts qualifying pauses between the first and last detected speech. The active minimum pause setting is 0.150 seconds. F0 and HNR use autocorrelation in overlapping 40 ms frames, with a configured 75-500 Hz pitch range.

Warnings cover short/quiet recordings, clipping, insufficient speech, and missing F0, even on successful runs. These lightweight estimates do not use Praat or WebRTC VAD, despite legacy configuration comments naming those tools.

All three tasks currently share the acoustic analysis core. Reading recordings also attach passage metadata in the record-and-assess flow; uploaded reading WAVs do not pass through that metadata-enrichment branch. The system does not currently implement transcription, verified words per minute, reading accuracy, word alignment, jitter/shimmer, or diagnostic speech classification.

Command-Line Operations

```
nexa speech
nexa speech list-microphones
nexa speech init-db
nexa speech record --task sustained_vowel --duration 5
nexa speech assess --task reading --duration 60
nexa speech analyze-speech recording.wav
nexa speech analyze-sustained-vowel vowel.wav
nexa speech analyze-reading reading.wav
nexa speech show-results
nexa speech --help
```

record records only; assess records and analyzes. analyze is also available as a general WAV-analysis command. Analysis commands accept participant, session ID, and baseline options. Standalone baseline comparison uses:

```
nexa speech analyze-speech baseline.wav --participant P001 --baseline
nexa speech analyze-speech followup.wav --participant P001 --session-id FOLLOWUP
nexa speech compare-baseline P001 FOLLOWUP
```

The standalone speech database and its baseline flags are separate from UI baseline selections. Prefer the UI baseline workflow for UI-managed sessions. Saved UI speech runs retain speech_recording.wav and speech_summary.json.

9. Storage, Privacy, and Baselines

Shared UI Data

nexa_data/	
nexa.sqlite3	Participants, sessions, runs, baselines
grip_calibration.json	Per-hand force calibration
staging/<modality_stamp>/	Temporary run outputs
raw/session_00001/<run>/	Saved artifacts and run_log.txt
reports/	Generated HTML reports

SQLite holds participant records, session metadata, run status, timestamps, metric JSON, operator notes, artifact locations, and baseline references. Raw files remain ordinary local files rather than SQLite BLOBs. Back up the database and raw directories together with calibration settings.

Standalone Service Data

- Speech: data/speech_analysis/, including recordings and database/nexa_speech.db.
- Shoulder: nexa_evaluations.db plus data/shoulder_monitor/ plot/CSV outputs.
- Eye: timestamped files under the working-directory-relative results/ folder.
- Grip: interactive graph and console output; JSON export depends on NEXA_OUTPUT_DIR.

UI execution can also produce service-specific database entries. Discarding a UI run does not roll back those independent records. Speech database audio paths may refer to staging locations removed after saving/discarding; use the copied WAV in the UI raw folder as the retained recording for saved UI runs.

Handling Sensitive Data

Recordings, identifiers, notes, and measurements can be sensitive. Use coded participant identifiers, obtain appropriate permission for capture/storage, restrict machine access, and apply an explicit retention and backup policy. No built-in user-account access control, database encryption, or automated retention service is established by the inspected implementation.

.gitignore excludes nexa_data/, databases, and most data/ content. This is not encryption and does not remove previously committed files from Git history. Standalone results/ and .venv-shoulder/ are not explicitly excluded in the current ignore file; check git status before committing. Do not commit real recordings, participant data, local environments, or hardware secrets.

Move or back up datasets with their directory structure intact. Stored absolute artifact paths may need adjustment after moving the repository or transferring a dataset to a different machine. The code does not provide a complete portable dataset migration workflow.

10. Troubleshooting, Limitations, and Maintenance

Common Failure Checks

- Command not found: rerun the platform launcher installer; on Windows open a new terminal to reload PATH.
- Missing Python package: install into the interpreter actually used by that service, not an unrelated global Python.
- Shoulder illegal instruction: use the tested dedicated environment; native CPU incompatibility is not fixed by downloading the model again.
- Shoulder does not start: focus its window, use Space/Start, select L/R, and ensure the required landmarks are visible.
- Grip has no graph updates: verify serial port/baud, exclusive port access, and newline-delimited right,left numeric readings.
- Grip sign or magnitude is wrong: tare unloaded, calibrate each hand with a known load, and test the reference force before assessment.
- Speech sample-rate error: inspect the selected microphone and actual supported rate; the recorder prefers 48000 Hz with fallback.
- Camera not detected: check model/overlay, ribbon orientation, and boot logs before changing Python packages.
- Camera frontend timeout: distinguish camera registration from successful frame delivery; test capture outside NEXA with no preview.
- Preview unavailable: use a real graphical desktop session; a placeholder DISPLAY value is not a valid display connection.
- Report does not open: inspect the generated HTML path and the system's default browser/file association.

Verification Checklist

Before collecting a new dataset, exercise one complete UI run per modality, including save, reopen in History, report generation, and baseline comparison. For speech, test each task and existing-WAV upload, and verify the retained WAV. For grip, test each hand independently at zero and known load. For shoulder, verify anatomical labels and the plot/CSV. For eye tracking, verify sustained frame delivery, pupil detection, sample intervals, and valid-trial reasons.

Run those checks on the actual target machine. Import success does not prove that a native library can initialize, and camera enumeration does not prove that image streaming works. No comprehensive automated hardware test suite is included in the inspected repository.

Known Product Boundaries

The countdown is estimated, some UI instructions retain older terminology, and only registered artifacts/metrics survive the normal UI collection flow. Baselines do not enforce algorithm-version or unit equivalence. Calibration is linear and does not compensate for creep, hysteresis, or temperature drift. Camera-derived measurements depend on lighting, framing, and reliable timing. Speech quality labels are not clinical judgments.

Source of Truth

This guide was assembled from the repository's scripts, configuration, service implementations, UI, storage layer, and README, together with the operator's reported hardware troubleshooting. It does not include private session contents. For exact current defaults, consult the relevant configuration/source file. README.md remains the shorter setup reference; ABOUT.md is the editable source for the accompanying program-guide PDF.